



RAFFLES INSTITUTION

2025 YEAR 6 TERM 3 TIMED PRACTICE

CANDIDATE
NAME

CLASS

25

COMPUTING

9569

Practical

2 hours

Additional materials: Electronic version of `attendance.json` data file
Electronic version of `LATE.CSV` data file
Electronic version of `STUDENT.CSV` data file
Electronic version of `word_game.py` file
Electronic version of `client.py` file
Electronic version of `server.py` file

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to 4 marks out of 70 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is 70.

FOR EXAMINER'S USE			
Task 1	Task 2	Task 3	
			TOTAL
			70

This document consists of **6** printed pages.

RAFFLES INSTITUTION
Mathematics Department

1 Name your Jupyter Notebook as `TASK1_<your class>_<your name>.ipynb`.

A teacher wants to keep track of instances where his students arrive late to college. To efficiently manage this data, he wants to create a database that records both the students' information and latecoming incidents. Two tables with the specified fields are to be created:

Student:

- `stu_id` – unique student ID, for example, 25076
- `name` – the student's full name
- `civics_class` – the class group that the student is in, for example, 25S02X
- `handphone` – the handphone number of the student, stored as a string

Latecoming:

- `date` – date of student's latecoming incident, for example, 2024-09-13
- `stu_id` – student ID of the latecoming student
- `reason` – reason for latecoming, as cited by the student

For each of the sub-tasks 1.1 to 1.3, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program code
```

Output:

Task 1.4 should be saved in a `.py` file with other files.

Task 1.1

Write a Python program that uses SQL code to create the database `college.db` with the two tables given, defining the appropriate primary and foreign keys for each table. [5]

Task 1.2

The teacher has downloaded data of students in his classes from the school portal as `STUDENT.csv`. Additionally, he also received recent student latecoming records from the Year Head, stored in another file `LATE.csv`.

Write a Python program to read in the data from each file and store them in the database. [4]

Task 1.3

Write a function `filter_latecomers` that takes the parameters `civics_class`, `min_latecomings`, `start_date`, `end_date` and returns a **list of strings**, which are the names (in alphabetical order) of students who belong to the specified class group and have been late at least the specified number of times within the date range.

The function should use SQL code to extract and filter the relevant data.

Test the function by printing the list of students from class 25S02Y who have at least 2 latecoming incidents in August 2024. [6]

Task 1.4

Create a **single-page** web application that allows the teacher to search for a student's latecoming records.

- A user that visits the page will be greeted with a form where he can input the name of a student to search for.
- If the student is found within the latecoming records, a table with orange cell borders should be displayed, showing the date and the latecoming reason for each latecoming record of the student.
- The records are to be sorted by date, with the latest latecoming record as the first row of data. An appropriate header row should be included in the table.
- Above the table, there should also be two lines of text, one line with the student's name and another showing the total number of latecoming instances.
- If no records are found for the student, the text "There are no latecoming records found for <student name>." should be shown on the page instead. No tables should be displayed.

The background colour of the page should be set as #2F8A7B. All texts on the page should be white and in Arial font.

Save the relevant files and subfolders in a folder named:

TASK1_4_<your name>.

[11]

2 Name your Jupyter Notebook as TASK2_<your class>_<your name>.ipynb.

The school is developing a flexible attendance monitoring system that captures daily attendance along with reasons for absence or presence. Due to the complexity of reasons (e.g., valid medical, unclear, school events), a NoSQL database is preferred, since certain absence requires additional information (e.g. MC will require a photo evidence). Attendance record for the 2 classes for a particular week in April is stored in the JSON file `attendance.json`.

The following fields are recorded:

- `stu_id`
- `name`
- `cg`
- `date`
- `status` ("Present" or "Absent")

These additional fields are recorded when only when status is "Absent":

- `reason_category` (e.g. Valid Family, Medical, Official Event, Pending Reason)
- `remarks`
- When the `reason_category` is "Medical", this field is required to provide the link to the image of the medical certificate.
 - `evidence_link`

Each JSON record stores a single day attendance record for a single student.

For each of the sub-tasks 2.1 to 2.5, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code
```

Output:

Task 2.1

Using Jupyter Notebook platform, write a Python program to import the information from the JSON file into a MongoDB database. Save the information under the `attendance` collection in the `college` database. Ensure that the collection **only** stores information from the JSON file.

Save your program code as TASK2_<your name>_<class>.ipynb [3]

Task 2.2

Write a function, `get_absentees_by_date(date)` that will display `stu_id`, `name`, `civics_class` and `reason_category` of students who are "Absent" on given date.

Display the information in tabular format, and display the total number of students absent that day. [5]

Task 2.3

Write a function, `num_absentees(start_date, end_date)` that will display the number of students who are absent from each class during the period between `start_date` and `end_date`.

Display the number of students absent from '2024-04-14' to '2024-04-16'. [6]

Task 2.4

Write a function `update_attendance(date, stu_id)` that allows the user to update a particular student's attendance on a particular date. You should check that an existing record for the particular student and date already exist before proceeding with the update.

To update the status to "Present", you will have ensure that fields like `reason_category`, `remarks` and `evidence_link` are removed if are present in the record.

To update the status to "Absent", you will have to ask the user for the `reason_category` and `remarks` values and update the record accordingly. If the `reason_category` is "Medical", you will also have to ask the user for the name of the image file to update the `evidence_link` field. [9]

Task 2.5

Create a menu to allow a user to continuously choose between listed options, and execute the corresponding code you wrote from **Task 2.2** to **Task 2.4**, until he/she chooses to quit. [3]

- 3 In a word game for young children, the program will randomly choose five four-letter words from its repository, scramble each word, and let the player input the correct arrangement of the words.

A working version of the game in `word_game.py` is provided for you to examine how the game runs.

Instead of using this version of the game, you are tasked to rewrite this game, **using Python sockets**, such that a server will play the role of generating 5 random words, and allow a client to guess the correct word. At the end of 5 attempts, the server will report the number of correct guesses.

Game from the **server** perspective:

```
Connected to: ('127.0.0.1', 54493)
```

Game from **client** perspective:

```
Welcome to the WORD GAME.
You will be given 5 scrambled words to answer ... here we go:

Enter the word you can form with "AEFL": LEAF
Correct

Enter the word you can form with "RHIC": RICH
Correct

Enter the word you can form with "EICP": PICE
Incorrect

Enter the word you can form with "RTEE": TREE
Correct

Enter the word you can form with "IEKL": KILE
Incorrect

You have answered 3 out of 5 correctly.
```

Using the templates, `server.py` and `client.py`, provided, write the server and client program codes for this Word game.

[14]